

AI Software Development Workspace — Functional Documentation

Version: 1.0 (Draft) **Stack:** React + TypeScript, Node.js, PostgreSQL, Redis, WebSockets, Docker, AI APIs, AWS

1. Product Overview

A project management system — Kanban board, sprints, docs, chat, GitHub-linked tasks — is the core product. AI is not the product; it's an **intelligent assistant that reduces the manual work of creating and maintaining PM data**. Anywhere a user would normally fill a structured form (create task, create requirement, and so on), they can instead describe it in plain language, and AI converts that into the structured fields. The user always reviews and confirms before anything is saved — AI drafts, the human decides.

Core value proposition: keep all the rigor of a proper PM tool (structured fields, traceability, reporting) while removing the tedium of typing them out field by field.

AI Design Philosophy

This principle applies across every AI-touched module in this document:

- **AI fills forms, it doesn't make PM decisions.** Priority, assignee, due date, scope — these stay human calls. AI's job is to correctly extract what the user already implied in their sentence, not to decide what *should* happen.
 - **Every AI output is a draft, never a save.** The user always sees the populated form and must explicitly confirm (e.g., "Create") before a record is persisted. Nothing is created or modified silently.
 - **Edit-in-place, not regenerate-from-scratch.** If a field is wrong, the user corrects that field directly rather than re-prompting the AI and hoping for a better result.
 - **Degrade gracefully.** If AI parsing is unavailable or low-confidence, the plain manual form is always there as a fallback — AI assistance is additive, never a blocker to getting work done.
-

2. Target Users & Roles

Role	Description	Primary Needs
Admin / Workspace Owner	Creates workspace, manages billing, integrations, members	Setup, access control, org-wide visibility
Product Manager / Lead	Writes requirements, plans sprints, tracks progress	AI requirement analysis, task generation, analytics
Developer	Picks up tasks, writes code, pushes to GitHub	Kanban, code snippets, AI code review, docs
Reviewer	Reviews PRs and AI-flagged issues	AI code review, GitHub integration
Viewer / Stakeholder	Read-only access to boards, docs, dashboards	Analytics, documentation

Roles should be enforced via **Role-Based Access Control (RBAC)** at the workspace and project level.

3. Functional Modules

3.1 Authentication & Workspace/Team Management

Purpose: Secure entry point and multi-tenant team structure.

- FR-1.1: Users can sign up/log in via email+password and OAuth (GitHub OAuth is high priority, given GitHub integration).
- FR-1.2: Users can create a **Workspace** (tenant) and invite members via email/link.
- FR-1.3: Admins can assign roles (Admin, PM, Developer, Reviewer, Viewer) per workspace or per project.
- FR-1.4: Users can belong to multiple workspaces and switch between them.
- FR-1.5: Session management via JWT (access + refresh token pattern); Redis used for refresh-token/session store and blacklist on logout.
- FR-1.6: Basic audit log of member additions/removals/role changes.

3.2 AI-Assisted Requirement Creation

Purpose: Let a PM describe a feature or problem in plain language and get a structured Requirement draft to review, instead of filling a multi-field form by hand.

- FR-2.1: User can paste/write free-form text describing a feature or problem (a paragraph, a meeting note, a voice-to-text dump).
- FR-2.2: AI parses the input and pre-fills a structured Requirement **draft**: problem statement, proposed scope, acceptance criteria, and a checklist of open

questions/ambiguities it couldn't infer.

- FR-2.3: The draft appears in the normal Requirement form — same fields as manual entry — and the user edits any field directly before proceeding.
- FR-2.4: The Requirement is only created when the user explicitly confirms; nothing is saved from the AI parse alone.
- FR-2.5: AI never silently fills a field it isn't confident about — it leaves it blank or flags it rather than guessing.
- FR-2.6: Requirement history is versioned — edits create a new version, not an overwrite.

3.3 AI-Assisted Task Creation

Purpose: Turn a plain-language task description into a filled-out Create Task form, instead of the user typing Title, Description, Priority, Assignee, Due Date, Story Points, Acceptance Criteria, and Labels one by one.

Primary flow — single task, quick create:

User types, e.g.:

"Create a high-priority task for Rahul to implement customer OTP login, due next Friday. It should include API integration and validation."

AI pre-fills the Create Task form:

Field	AI-filled value
Title	Implement Customer OTP Login
Description	Implement OTP-based login for customers, including API integration and validation
Assignee	Rahul
Priority	High
Due Date	14 Aug 2026 (resolved from "next Friday")
Acceptance Criteria	✓ OTP can be requested · ✓ OTP validation works · ✓ Invalid OTP shows an error · ✓ Expired OTP is rejected

User reviews the pre-filled form, edits any field if needed, then clicks **Create** — the same flow as manual creation, just with the typing done for them.

- FR-3.1: User can type a natural-language task description from a "Quick Create" entry point on the Kanban board, or from within a Requirement.
- FR-3.2: AI extracts and pre-fills: title, description, assignee (matched against workspace members by name), priority, due date (relative dates like "next Friday")

resolved against the current date), story points (if inferable, else left blank), acceptance criteria (as a checklist), and labels.

- FR-3.3: The pre-filled Create Task form is shown before anything is saved — identical form to manual task creation, just pre-populated.
- FR-3.4: If AI can't confidently resolve a field (e.g., an assignee name that doesn't match anyone in the workspace), that field is left blank rather than guessed, and flagged for the user.
- FR-3.5: Nothing is created until the user clicks Create.

Secondary flow — batch generation from a Requirement (optional, builds on the same capability):

- FR-3.6: From an accepted Requirement, user can trigger "Suggest Tasks" to get multiple task drafts at once (e.g., split by frontend/backend/infra), each pre-filled the same way as the single-task flow.
 - FR-3.7: User accepts, edits, merges, or discards each suggested task individually before any are created.
 - FR-3.8: Tasks created this way are linked back to the parent Requirement for traceability.
-

3.4 Kanban Board

Purpose: Core task-tracking surface.

- FR-4.1: Board with configurable columns (default: Backlog, To Do, In Progress, In Review, Done).
 - FR-4.2: Drag-and-drop card movement between columns, persisted in real time.
 - FR-4.3: Card detail view: description, assignee(s), labels, priority, linked Requirement, linked PR (from GitHub integration), comments, attachments.
 - FR-4.4: Filtering/search by assignee, label, priority, sprint.
 - FR-4.5: Swimlanes by assignee or epic (optional toggle).
 - FR-4.6: WIP limits per column (optional, configurable per project).
-

3.5 Sprint Planning

Purpose: Time-boxed planning layer on top of the board.

- FR-5.1: User can create Sprints with start/end dates and a goal statement.
- FR-5.2: Tasks can be assigned to a Sprint from the Backlog (drag or bulk-select).
- FR-5.3: Sprint view shows total points/tasks committed vs. capacity (basic capacity = team size × average velocity, editable).
- FR-5.4: Burndown chart for the active sprint (see Analytics Dashboard, 3.12).
- FR-5.5: Sprint close-out: incomplete tasks auto-prompt to move to next sprint or back to backlog.

- FR-5.6: Sprint retrospective notes field (freeform, linked to Rich Text Documentation module).
-

3.6 Rich Text Documentation

Purpose: Notion-style docs for specs, wikis, meeting notes.

- FR-6.1: Block-based rich text editor (headings, lists, code blocks, tables, images, embeds).
 - FR-6.2: Docs organized in a nested page hierarchy per workspace/project.
 - FR-6.3: Docs can be linked to Requirements, Tasks, or Sprints (bi-directional linking).
 - FR-6.4: Version history with restore-to-previous-version.
 - FR-6.5: Slash-command style block insertion (`/table`, `/code`, `/image`) for fast authoring.
 - FR-6.6: Full-text search across all docs in a workspace.
-

3.7 Code Snippets

Purpose: Shareable, syntax-highlighted code fragments outside of full GitHub context (quick references, examples, reusable utils).

- FR-7.1: Users can create snippets with language selection and syntax highlighting.
 - FR-7.2: Snippets are taggable and searchable.
 - FR-7.3: Snippets can be embedded inside Docs (3.6) or Chat messages (3.8).
 - FR-7.4: Snippet versioning (edit history).
 - FR-7.5: Optional: "run" for a small set of safe languages via sandboxed execution (stretch goal, not MVP — see Section 6).
-

3.8 Team Chat

Purpose: Lightweight in-context communication tied to the workspace.

- FR-8.1: Channel-based chat (per project or custom channels) plus direct messages.
 - FR-8.2: Messages support rich text, code snippets, and file attachments.
 - FR-8.3: Messages can be linked/referenced from a Task or Doc ("mentioned in chat").
 - FR-8.4: @mentions trigger in-app + email notifications.
 - FR-8.5: Message history is persisted and searchable.
-

3.9 Real-Time Collaboration

Purpose: Cross-cutting capability that makes the Board, Docs, and Chat feel "live." This is infrastructure serving other modules rather than a standalone screen.

- FR-9.1: Kanban card moves reflect instantly for all connected users (WebSocket broadcast).

- FR-9.2: Docs support concurrent multi-cursor editing with presence indicators (who's viewing/editing).
 - FR-9.3: Chat messages deliver in real time with typing indicators.
 - FR-9.4: Presence system: online/offline/away status per user, workspace-wide.
 - FR-9.5: Conflict resolution strategy for simultaneous doc edits (recommend operational transform or CRDT library rather than hand-rolled — flag as an architecture decision, see Section 7).
-

3.10 AI Code Review

Purpose: Automated first-pass review triggered by GitHub PR activity.

- FR-10.1: On PR open/update (via GitHub webhook), AI reviews the diff.
 - FR-10.2: AI posts a structured summary: potential bugs, style/convention deviations, missing test coverage signals, security concerns (e.g., hardcoded secrets, obvious injection risks).
 - FR-10.3: AI comments are posted either as PR comments (via GitHub API) or surfaced inside the workspace UI (or both — configurable).
 - FR-10.4: Developer can mark AI comments as "Resolved" / "Not applicable" / "Fix applied."
 - FR-10.5: Review runs are scoped to changed files only (not full repo) for cost and relevance.
 - FR-10.6: Rate/queue AI review runs — avoid re-running on every single push if a PR is updated rapidly (debounce).
-

3.11 GitHub Integration

Purpose: Connect planning to actual code activity.

- FR-11.1: OAuth-based GitHub App connection per workspace, scoped to selected repos.
 - FR-11.2: Link a Task to a branch/PR (manual link or auto-detect via branch naming convention, e.g., `task-123-description`).
 - FR-11.3: PR status (open/draft/merged/closed) reflected on the linked Task card.
 - FR-11.4: Commits referencing a task ID (e.g., in commit message) auto-link to that task.
 - FR-11.5: Webhook listener for push, PR, and review events to keep task status in sync.
 - FR-11.6: Optional: auto-move Task to "In Review" when a linked PR opens, and to "Done" when merged (configurable automation).
-

3.12 Analytics Dashboard

Purpose: Visibility for PMs/leads into team and project health.

- FR-12.1: Sprint burndown/burnup chart.

- FR-12.2: Velocity trend across past sprints.
 - FR-12.3: Cycle time (time from "In Progress" to "Done") per task, averaged.
 - FR-12.4: Task distribution by assignee, label, and status (current snapshot).
 - FR-12.5: AI usage stats (requirements analyzed, tasks generated, code reviews run) — useful for showing product value.
 - FR-12.6: Exportable reports (CSV/PDF) for a given sprint or date range.
-

4. Non-Functional Requirements

- **Performance:** Kanban and Chat updates should reflect for other users within ~300ms under normal load (WebSocket-driven).
 - **Scalability:** Redis for pub/sub (WebSocket fan-out across multiple Node instances) and caching hot reads (board state, presence).
 - **Security:** RBAC enforced server-side on every request, not just UI-hidden; secrets (GitHub tokens, AI API keys) encrypted at rest; webhook payloads verified via signature.
 - **Reliability:** AI API calls (requirement analysis, task generation, code review) must have timeout + retry + graceful degradation (don't block the UI if AI is slow/down).
 - **Data isolation:** Strict multi-tenancy — workspace data must never leak across tenants (enforce at query layer, not just app layer).
 - **Deployment:** Dockerized services, deployed to AWS (e.g., ECS/EKS + RDS Postgres + ElastiCache Redis + S3 for attachments).
 - **Observability:** Basic logging/metrics per service; AI call latency and failure rate tracked separately since it's a dependency on a third party.
-

5. High-Level Data Model (Core Entities)

```
Workspace — has many — Users (via WorkspaceMember, with Role)
Workspace — has many — Projects
Project — has many — Requirements
Requirement — has many — Tasks (AI-generated or manual)
Task — belongs to — Sprint (optional)
Task — belongs to — Column/Status
Task — linked to — PullRequest (via GitHub integration)
Project — has many — Docs (nested tree via parent_doc_id)
Project — has many — Channels — has many — Messages
Message / Doc — can embed — Snippet
PullRequest — has many — AIReviewComments
Sprint — has many — Tasks, has one — RetrospectiveDoc
```

This is intentionally high-level — actual schema (columns, indexes, junction tables for RBAC) should be worked out per module during implementation.

6. Suggested MVP Phasing

Given the surface area here, building all 12 modules at once is a recipe for a half-finished everything. A phased build keeps each stage independently demoable and testable.

Phase 1 — Core Loop (no AI yet) Auth & Workspace, Kanban Board, basic Sprint Planning. Prove the core PM loop works end-to-end with manual task creation.

Phase 2 — AI-Assisted Creation AI-Assisted Task Creation (single-task quick create, FR-3.1-3.5 — the primary use case) wired into the Phase 1 board, then AI-Assisted Requirement Creation. Batch task generation from a Requirement (FR-3.6-3.8) is the same underlying capability applied to multiple drafts at once, so it's low-risk to add after the single-task flow is solid rather than building both in the first pass.

Phase 3 — Collaboration Surfaces Rich Text Docs, Code Snippets, Team Chat, Real-Time Collaboration (WebSockets). These are independent of AI and can be built in parallel once core loop is stable.

Phase 4 — GitHub + AI Code Review GitHub Integration first (webhooks, PR linking), then AI Code Review on top of it — review depends on integration being solid.

Phase 5 — Analytics Dashboard Naturally comes last since it aggregates data generated by everything above.

7. Open Questions / Decisions Needed

- Which AI provider(s) for requirement analysis / task generation / code review — one model for all three, or task-specific models?
- Real-time doc editing: CRDT library (e.g., Yjs) vs. simpler last-write-wins with conflict warning — affects Phase 3 scope significantly.
- Is code snippet "execution" (FR-7.5) in scope at all, given sandboxing complexity/cost? Recommend deferring past MVP.
- GitHub App vs. OAuth App for the integration — GitHub App gives finer-grained repo permissions and is generally the better fit here.
- Self-hosted vs. managed Postgres/Redis on AWS (RDS/ElastiCache) — affects Docker/deployment setup in Section 4.

This document covers functional scope only. Technical architecture (API contracts, DB schema, WebSocket event design) should follow as a separate design doc once Phase 1 scope is locked.